Jetpack Compose — Cheat Sheet

From @Composable basics through state, recomposition, side effects, and deep internals · Interview-prep flavor

26 sections | Easy → Advanced | Internals demystified | 2026 Edition

About this sheet: a personal high-density study aid for Jetpack Compose, written entirely in plain English with original examples and standard idiomatic patterns. Covers the full spectrum: from "what is `@Composable`" through state, recomposition, side effects, modifiers, the three phases (composition / layout / drawing), animations, navigation, accessibility, testing — and the deep internals (compiler transforms, slot table, snapshot system, smart recomposition). Includes a "★ Internals — what's NOT magic" demystification section, a comprehensive do's-and-don'ts list, and a one-page recall card. Pair with an internals book like *Jetpack Compose Internals* by Jorge Castillo for the next level of depth.

1. The mental model — Compose in 60 seconds

Jetpack Compose is a **declarative** Kotlin-DSL UI toolkit. You describe *what* the UI should look like for a given state; Compose figures out *how* to update the screen when state changes.

	OLD VIEW SYSTEM (XML + KOTLIN/JAVA)	JETPACK COMPOSE
Style	Imperative: mutate views directly	Declarative: UI = f(state)
Source of UI definition	XML + code merging via findViewById/binding	One Kotlin file with `@Composable` functions
State sync	Manual (you push state into views, easy to drift)	Automatic (state change → recomposition)
Custom UI	Custom View / ViewGroup (heavy)	Custom `Composable` + `Layout` (light)
Animation	Animator + ObjectAnimator	`animate*AsState`, `Transition`, integrated
Live preview	Layout Inspector after launch	`@Preview` in Android Studio, instant

Three things to internalize:

1. **UI is a function of state.** Same state in → same UI out. Never mutate widgets directly.
2. **Recomposition is cheap, not free.** Compose reruns Composable functions when state they read changes. The runtime tries to skip subtrees that haven't changed (smart recomposition), but writing skippable code is on you.

3. **Composables are short-lived.** The function body re-runs many times. Side effects must go in side-effect APIs, not the function body itself.

```
@Composable
fun Greeting(name: String) {
    Text("Hello, $name!")
}
// Compose runtime tracks the read of `name`. Any change → just this Composable re-runs.
```

2. Composable functions — what `@Composable` actually does

`@Composable` is not a marker. It changes the function's type and contract. After the Compose Compiler plugin runs, your function silently gets a hidden `Composer` parameter, plus extra bookkeeping parameters (more in §6 internals).

```
// You write:
@Composable
fun NamePlate(name: String) { Text(name) }

// Compiler emits roughly:
fun NamePlate(name: String, $composer: Composer, $changed: Int) { ... }
```

That hidden `Composer` parameter is what makes the magic work. Composables can **only be called from other Composables** (or special entry points like `setContent`) — because only then is a Composer in scope to hand off.

The five rules of well-behaved Composable functions

RULE	WHAT IT MEANS	WHY IT MATTERS
Calling context	Can only be invoked where a Composer is in scope	The runtime needs the Composer to track this call's position in the tree
Idempotent	Same inputs produce same effect on the tree, every time	So Compose can skip / replay safely
Free of side effects	The body itself doesn't observably change app state outside Compose	Side effects belong in `LaunchedEffect` / `DisposableEffect` / `SideEffect`
Restartable	Compose may re-run the function many times when its inputs change	Your code must tolerate repeated execution without breaking
Fast	Body should be cheap (microseconds), not do real work	Recomposition happens on the UI thread; slow Composables = jank

Bonus rule: Composables are also **positionally memoized** — Compose tracks each call's position in the tree, so two calls to the same Composable at different positions are independent.

Composable functions are "colored" — like `suspend`

Just like `suspend`, `@Composable` partitions the function space. A regular function can't call a Composable. A Composable can only be called from another Composable (or a builder like `setContent { }` or `composable { }`). This is enforced at compile time. Same idea as Kotlin's "colored" suspend functions.

3. State — `mutableStateOf`, `remember`, `rememberSaveable`

State is what makes recomposition meaningful. Compose tracks every State read inside Composables; when that State's value changes, the readers are scheduled for recomposition.

```
@Composable
fun Counter() {
    var count by remember { mutableStateOf(0) }   // State of Int
    Button(onClick = { count++ }) {                  // write — State updates
        Text("Clicks: $count")                       // read — this Text recomposes when count changes
    }
}
```

HELPER	SURVIVES RECOMPOSITION?	SURVIVES CONFIG CHANGE?	SURVIVES PROCESS DEATH?
`remember { ... }`	Yes	No	No
`rememberSaveable { ... }`	Yes	Yes (via Bundle)	Yes (via Bundle)
State in ViewModel	Yes	Yes	No (use `SavedStateHandle` for that)

The four State factories you'll use

- `mutableStateOf(initial)` — basic `MutableState<T>`. Use `by` delegate for property-style access.
- `mutableIntStateOf(0)` / `mutableLongStateOf` / etc. — primitive variants, avoid boxing in hot paths (lists, animations).
- `mutableStateListOf()` / `mutableStateMapOf()` — observable lists/maps; mutating them triggers recomposition of readers.
- `derivedStateOf { computed }` — wraps a derived value so recomposition fires only when the result actually changes (see §10).

> **Don't:** use plain `var x = 0` inside a Composable for mutable UI state. Updating it does *not* trigger recomposition. Always wrap mutable UI state in `State`.

What `by` actually does

`var count by remember { mutableStateOf(0) }` uses Kotlin property delegation. The delegate's `getValue` reads `state.value`, the `setValue` writes it. So `count++` compiles to `state.value = state.value + 1`. Pure syntactic sugar — the State object is still what Compose tracks.

4. Recomposition — what it is, when it happens

Recomposition = re-running a Composable function with new inputs. It happens because something the Composable read (a State value, a CompositionLocal, or one of its parameters) changed.

What gets recomposed?

- Compose tracks *which* Composable read which State.
- When State changes, Compose schedules the smallest enclosing **RecomposeScope** (typically the nearest restartable Composable function) for re-execution.
- Other parts of the tree are **skipped** if their inputs haven't changed and their parameters are **stable** (see §9 for stability).

```
@Composable
fun Counter() {
    var a by remember { mutableStateOf(0) }
    var b by remember { mutableStateOf(0) }
    Column {
        Text("A: $a")            // reads a → only this Text re-runs when a changes
        Text("B: $b")            // reads b → only this Text re-runs when b changes
        Button(onClick = { a++ }) { Text("+A") }
        Button(onClick = { b++ }) { Text("+B") }
    }
}
```

Tapping `+A` only re-runs the first Text. The Column, Buttons, and the second Text are skipped. This is the heart of "smart recomposition."

> **Tip — Layout Inspector:** turn on "Show recomposition counts" in Android Studio's Layout Inspector to see exactly which Composables recompose, how often, and how many were skipped. Best perf-debugging tool you have.

5. State hoisting — the most important pattern

"Hoist state" = move state out of a Composable to its caller. The Composable becomes **stateless** and accepts `(value, onValueChange)` as parameters. Now it's reusable, testable, previewable, and the caller controls behavior.

```
// Bad: state hidden inside, no way for caller to read or react
@Composable
fun NameInput() {
    var name by remember { mutableStateOf("") }
    TextField(name, onValueChange = { name = it })
}

// Good: hoisted
@Composable
fun NameInput(name: String, onNameChange: (String) -> Unit) {
    TextField(name, onValueChange = onNameChange)
}

// Caller owns state — can validate, persist, share with other Composables
var name by rememberSaveable { mutableStateOf("") }
NameInput(name, onNameChange = { name = it.trim() })
```

Where should state live?

- **UI-only ephemeral** (focus state, expanded/collapsed, scroll position) → in the Composable via `remember`
- **Survives rotation but not process death** → ViewModel (`StateFlow` + `collectAsStateWithLifecycle`)
- **Survives process death** → `rememberSaveable` for small primitives, `SavedStateHandle` in ViewModel for VM-scoped, Room/DataStore for durable
- **Shared across Composables** → hoist to lowest common ancestor; for app-wide, ViewModel

Rule of thumb: hoist state to the *lowest common ancestor* of all Composables that need it. Don't hoist higher than necessary (creates unnecessary recompositions).

★ Internals — what's NOT magic (the engine demystified)

People treat Compose as magic. It isn't. Every "magic" behavior is a small piece of plain Kotlin code. Once you understand the engine, the framework's quirks make sense and you stop fighting it.

The three big building blocks

1. **Compose Compiler** — a Kotlin compiler plugin that rewrites every `@Composable` function to inject the `Composer` and add tracking metadata.
2. **Compose Runtime** — the in-memory data structure (the *slot table*) that records what was composed, plus the *Composer* that manages reading/writing it.
3. **Snapshot System** — the read-tracking mechanism that knows which Composable read which State, so changes can invalidate the right RecomposeScopes.

1. What the Compose Compiler does to your @Composable function

> **Looks like magic:** annotating a function with `@Composable` somehow adds Compose superpowers.
> **Actually:** the Compose Compiler plugin transforms each Composable in three ways:

1. **Injects hidden parameters**: a `Composer` instance + a `$changed: Int` bitmask + a `$default: Int` bitmask (for default arguments).
2. **Wraps the body** in `composer.startRestartGroup(key)` and `composer.endRestartGroup()?.updateScope { ... }`. This creates a *RecomposeScope* — the unit Compose can re-execute on its own.
3. **Adds skip logic**: at the top of the function, check if all parameters are unchanged AND stable. If yes, call `composer.skipToGroupEnd()` and return without running the body — that's **recomposition skipping**.

```
// You write:
@Composable fun Greeting(name: String) { Text("Hello $name") }

// Compiler emits roughly (simplified):
fun Greeting(name: String, $composer: Composer, $changed: Int) {
    $composer.startRestartGroup(0xABCD)
    var dirty = $changed
    if ($changed and 0b1 == 0)
        dirty = dirty or if ($composer.changed(name)) 0b100 else 0b10

    if (dirty and 0b11 != 0b10 || !$composer.skipping) {
        Text("Hello $name", $composer, 0)
    } else {
        $composer.skipToGroupEnd()                // ← skip!
    }

    $composer.endRestartGroup()?.updateScope { c, _ ->
        Greeting(name, c, $changed or 1)        // how to re-run if invalidated
    }
}
```

2. The slot table — Compose's in-memory tree

> **Looks like magic:** Compose somehow remembers state across recompositions even though Composables are just function calls.
> **Actually:** the Composer maintains a **slot table**, which is a flat array (gap buffer) representing the tree of all Composable calls and their stored values. Every `remember`, every `State`, every parameter, every group boundary — they all become *slots* in this array.

How a Composable reads/writes the slot table:

- On first composition, Compose *writes* a new entry: a group token, the parameters, the `remember`ed values.
- On recomposition, Compose *reads* the entry at the same position, comparing parameters. If unchanged, skip. If changed, update.
- The flat-array layout means traversal is cache-friendly. The "gap buffer" lets Compose insert/remove items in the middle efficiently.

3. Positional memoization — why `remember` works

> **Looks like magic:** calling `remember { generateExpensiveValue() }` twice doesn't run the lambda twice — even though it's a normal Kotlin function call.
> **Actually:** `remember` stores its result at the current position in the slot table. On re-execution, it asks the Composer "is there already a value at this position with these inputs?" If yes, returns it without invoking the lambda. The "position" is determined by the call site in the source code (Composer's source-location tracking), not by argument identity. *That's why two `remember` calls in the same function produce two independent memoized values.*

```
@Composable fun Demo() {
    val a = remember { compute(1) }    // stored at position P1
    val b = remember { compute(2) }    // stored at position P2
    // Even if compute(1) == compute(2), these are DIFFERENT memoized slots.
}
```

4. The snapshot system — how reads of State get tracked

> **Looks like magic:** reading `state.value` inside a Composable somehow registers that "this Composable depends on this State."
> **Actually:** when Compose enters a RecomposeScope, it sets a thread-local `SnapshotObserver`. Every `state.value` read goes through the State's `readable` method, which calls back to the observer: "scope X read state Y." The observer records this in a (state → scopes) map. When state.value is later set to a new value, the observer fires the recomposition for every scope that read it.

This is why Composable functions must read State *during* their execution — not later, not in a callback. If the read happens outside the observer's context, it's not tracked, and recomposition won't trigger.

5. The Snapshot itself — MVCC for State

Snapshots aren't just a tracking mechanism. Each Snapshot is a *versioned view* of all State at a point in time. This enables:

- **Concurrent reads**: composition can read State while another thread is mutating, without locks — each side sees a consistent snapshot.
- **Atomic mutation**: `Snapshot.takeMutableSnapshot()` lets you mutate State in isolation, then atomically `apply()` all changes together (or abandon them).
- **Conflict detection**: when applying, Compose checks for write conflicts and can merge/reject.

It's basically Multi-Version Concurrency Control (MVCC), the same idea databases use.

6. Smart Recomposition — why most of the tree is skipped

> **Looks like magic:** only the affected Composable re-runs; thousands of others are skipped.
> **Actually:** three things combine.

1. **Per-State subscriber tracking** (above) → Compose knows exactly which RecomposeScopes depend on the changed State.

2. **Per-parameter change check** via `$changed` bitmask → on entry to a Composable, Compose compares old vs new parameters bit-by-bit; unchanged + stable means "skip the body."
3. **Slot-table replay** → for skipped groups, Compose just advances the slot table pointer without executing — the values are already there.

7. What is a RecomposeScope?

A RecomposeScope is the smallest unit of Composition that Compose can re-run independently. By default, every restartable Composable function is its own scope. Inline functions and lambdas with the `@Composable` annotation may or may not get their own scope (the compiler decides based on whether they have parameters and whether they're stable).

Practical impact: **if you read State inside a Composable, the smallest enclosing scope is invalidated**. So defer reads as far down the tree as possible — read inside a leaf, not the root.

8. Lambdas inside Composables get memoized

> **Looks like magic:** passing `onClick = { vm.click() }` doesn't cause infinite recompositions even though a new lambda is created on every call.
> **Actually:** the Compose Compiler does *lambda memoization*. If a lambda has no captured variables, or all its captures are stable and unchanged, the compiler wraps it in a `remember`-equivalent slot and reuses the same instance. So your child Composable receives the same lambda reference and stays skippable.

9. The Recomposer — the orchestrator

The **Recomposer** is a singleton coroutine running on the AndroidUiDispatcher (Compose's UI dispatcher). It receives invalidation signals from State changes, batches them per frame, and triggers recomposition on the affected scopes when the next frame is requested. This is why state changes are coalesced — multiple mutations in the same frame produce a single recomposition.

10. `setContent { }` — what actually happens at the entry point

- An `AbstractComposeView` is added to the View hierarchy.
- A `Composition` is created, attached to a `Recomposer`.
- The first composition runs your `setContent { }` block, building the slot table and creating the LayoutNode tree.
- The LayoutNode tree integrates with the View system's measure/layout/draw pass.

> **Takeaway:** Compose is a small, elegant runtime — a slot table, a Composer, and a snapshot system. Once you understand these three, "advanced" Compose stops feeling mysterious. Reading the kotlinx Compose source becomes routine, not intimidating.

★ Composition vs Recomposition — what they mean, how Google built it, how to skip work

People throw "composition" and "recomposition" around like the same thing. They're not. Understanding the difference unlocks performance work and impresses interviewers.

The vocabulary (clear definitions)

TERM	WHAT IT MEANS
Composition (the noun)	The in-memory representation of your UI tree at a moment in time. It's a data structure — specifically, a slot table — that records which Composables ran with which parameters, what they `remember`'d, and how they nest.
Composition (the verb / "composing")	The act of running a Composable function for the *first time*. Adds new entries to the slot table.
Recomposition	The act of running a Composable function *again* with new inputs. Updates existing entries in the slot table at the same positions.
Initial composition	The first composition for a given scope — happens once when it enters the tree.
Skipping	Compose decides "all inputs unchanged AND stable, no need to re-run the body" and advances the slot table pointer past this Composable's entries without executing.
RecomposeScope	The smallest unit Compose can re-run independently. By default, every restartable @Composable function creates one.

So: a Composable is **composed once** when it first appears, and **recomposed many times** as state changes. Both go through the same compiler-generated code path — the difference is whether the slot table already has entries at this position.

How Google built this — the inheritance from existing computer science

Compose isn't magic from outer space. Every piece of it has roots in older, well-understood technology. Google's contribution was integrating these proven ideas into a coherent UI framework with first-class Kotlin compiler support — not invented-here. Here's the genealogy:

COMPOSE CONCEPT	WHERE IT CAME FROM
Slot table (gap buffer array)	Text editors since the 1970s. Emacs uses gap buffers for the same insert/delete-in-the-middle efficiency reason. Compose stores its tree as a flat array; the "gap" lets it shift entries when the tree mutates.
Positional memoization	Functional programming + React's reconciliation idea. React diffs a virtual DOM tree; Compose tracks position in a flat slot table — same goal (identify "this element didn't move"), different data structure.
Snapshot system (MVCC)	Multi-Version Concurrency Control from databases since the 1980s — Postgres, Oracle, etc. use MVCC for "consistent reads while writers are mutating." Compose uses the exact same idea for State, so composition can read state on the UI thread while another coroutine mutates it.
Compiler plugin / IR transformation	Same Kotlin IR (Intermediate Representation) pipeline that `suspend` functions go through. Compose Compiler is essentially "suspend-style code transformation, but for UI state." The same Kotlin compiler team that built suspend built Compose Compiler.
State observation (snapshot reads)	MobX, Vue's reactivity system, RxSwift Observables, Knockout.js. Same idea: track which code reads which state by intercepting the read.
Modifier chain (decorator pattern)	Gang of Four Decorator pattern (1994). Identical structurally to OkHttp's interceptor chain, Servlet filters, middleware in any web framework.
Composer (state machine)	Coroutines' Continuation pattern — both are state-machine-driven control flow. Calls like `startRestartGroup` / `endRestartGroup` mirror coroutines' `$label` + `resumeWith`.
Recomposer (frame batching)	Choreographer + VSYNC, which Android has had since API 16. Compose hooks into Choreographer's frame callbacks — when a state change happens, the Recomposer batches all invalidations and applies them at the next frame boundary.
Stability inference	Type system theory — similar in spirit to Rust's borrow checker reasoning about immutability and aliasing.
Single-shot execution model	React's "render is just a function" idea (2013). Compose pushed it further by making the function literally re-runnable at the granularity of a RecomposeScope.

The clever choice — flat slot table vs virtual DOM tree

The non-obvious decision: Compose uses a **flat slot table** instead of a virtual DOM tree like React.

- **React approach**: each render builds a new tree of plain objects, then diffs against the previous tree. Allocation per render + tree-diff cost.
- **Compose approach**: a single flat array (gap buffer) where each slot is a `(key, value)` pair. Recomposition walks the array and updates in place. Zero allocation per recomposition. Cache-friendly memory access.

Trade-off: Compose can't ask "what would this UI look like with totally new state?" without actually running the code. React can compute "next tree" without committing it. For mobile UI, the Compose trade-off is the right one — way faster on memory-constrained devices.

What's actually inside the slot table for a simple Composable

```
@Composable
fun Counter() {
    var count by remember { mutableStateOf(0) }
    Text("Clicks: $count")
    Button(onClick = { count++ }) { Text("+") }
}
```

The slot table contains roughly:

[restart-group: Counter] [remember-group: count] [State<Int>(0)] [restart-group: Text] [parameter: "Clicks: 0"] [LayoutNode(text = ...)] [restart-group: Button] [parameter: lambda#1] [LayoutNode(button = ...)] [restart-group: Text] [parameter: "+"] [LayoutNode(text = ...)] [end-group]
Each entry has a position (index in the array). When recomposition happens, the Composer walks the table from the start of the affected RecomposeScope. If the next entry's parameters match what's stored, it skips. Otherwise it updates in place. Most of recomposition is just walking past entries that haven't changed.

What "smart recomposition" actually means (three optimizations combined)

1. **Targeted invalidation** — when state changes, only the smallest enclosing RecomposeScope that read it gets queued for re-execution. Other scopes don't know anything happened.
2. **Stability check at function entry** — generated code at the top of every restartable Composable checks the `$changed` bitmask. If all params are unchanged AND stable, the body is skipped entirely.
3. **Lambda memoization** — the Compose Compiler caches lambdas with stable captures so child Composables receive the same lambda reference and stay skippable.

Together: a feed with 100 items where only item #5's data changed re-runs only item #5's restart group. Items 1-4 and 6-100 are skipped — the slot-table walker advances past their entries without execution. *That's the whole performance story in one paragraph.*

8 common causes of unnecessary recompositions — with code fixes

This is where Compose performance interviews drill. Each pattern: why it happens + the fix.

1. Reading the whole UiState in a parent Composable

Bad:

```
@Composable
fun Header(vm: MyViewModel) {
    val state by vm.uiState.collectAsStateWithLifecycle()
    Text("Welcome ${state.userName}")        // recomposes on EVERY field change
    LotsOfOtherStuff(state)                  // also recomposes on every field change
}
```

Why: `Header` reads the whole `state` object, so it recomposes whenever any field of `state` changes — even unrelated ones like scroll position.

Good:

```
@Composable
fun Header(vm: MyViewModel) {
    val userName by remember(vm) {
        derivedStateOf { vm.uiState.value.userName }
    }
    Text("Welcome $userName")              // recomposes ONLY when userName changes
}

// Or — better — expose individual flows from VM:
val userName: StateFlow<String> = uiState.map { it.userName }.stateIn(...)
```

2. Inline lambdas creating new instances per recomposition

Bad:

```
@Composable
fun Parent(items: List<Item>) {
    LazyColumn {
        items(items) { item ->
            ItemRow(item, onClick = { handleClick(item.id) })   // new lambda per recomp
        }
    }
}
```

Why: The lambda `{ handleClick(item.id) }` captures `item` — a different instance per row. Each recomposition creates a new lambda; ItemRow sees "different" callback and can't skip.

Good:

```
@Composable
fun Parent(items: List<Item>, onClick: (String) -> Unit) {
    LazyColumn {
        items(items, key = { it.id }) { item ->
            ItemRow(item, id = item.id, onClick = onClick)      // stable lambda
        }
    }
}

// In ItemRow:
@Composable
fun ItemRow(item: Item, id: String, onClick: (String) -> Unit) {
    Row(Modifier.clickable { onClick(id) }) { ... }             // click closure inline is OK
}                                                                // because clickable's modifier handles it
```

3. Unstable list / collection parameters

Bad:

```
@Composable
fun UserList(users: List<User>) { ... }
// kotlin.collections.List is UNSTABLE (could be MutableList).
// UserList always recomposes when its parent does, even if users hasn't changed.
```

Good:

```
// Add: implementation("org.jetbrains.kotlinx:kotlinx-collections-immutable:0.3.7")

@Composable
fun UserList(users: ImmutableList<User>) { ... }
// ImmutableList is annotated @Immutable. UserList becomes skippable.

// In ViewModel, expose as ImmutableList:
val users: StateFlow<ImmutableList<User>> = userFlow.map { it.toImmutableList() }
    .stateIn(scope, WhileSubscribed(5_000), persistentListOf())
```

4. Reading scroll position in the Composition phase

Bad:

```
@Composable
fun Screen(listState: LazyListState) {
    val position = listState.firstVisibleItemIndex     // changes every scroll tick
    Text("Item: $position")                            // recomposes every scroll pixel
    BigList(listState)
}
```

Good:

```
@Composable
fun Screen(listState: LazyListState) {
    val showHeader by remember {
        derivedStateOf { listState.firstVisibleItemIndex > 5 }
    }
    AnimatedVisibility(showHeader) { Header() }             // recomposes only on threshold flip
    BigList(listState)
}
```

5. Passing `State<T>` instead of `T`

Bad:

```
@Composable
fun NameDisplay(name: State<String>) {
    Text(name.value)                          // reads inside
}
```

Why: The caller passes the same State reference; the child reads `.value` inside. Now the child's reads invalidate this scope on every state change. By passing the State down, you've moved the read from a leaf to a parent — inverting what skipping wants.

Good:

```
@Composable
fun NameDisplay(name: String) {
    Text(name)                              // receives plain value
}

// Caller reads State and passes value:
val name by vm.name.collectAsStateWithLifecycle()
NameDisplay(name)
```

6. Animations driving high-frequency value reads in Composition

Bad:

```
@Composable
fun RotatingIcon(angle: Float) {
    Icon(
        modifier = Modifier.rotate(angle),    // reads angle in COMPOSITION phase
        imageVector = Icons.Default.Star,
        contentDescription = null
    )
    // → recomposes EVERY animation frame
}
```

Good:

```
@Composable
fun RotatingIcon(angle: Float) {
    Icon(
        modifier = Modifier.graphicsLayer { rotationZ = angle },   // reads in DRAWING phase
        imageVector = Icons.Default.Star,
        contentDescription = null
    )
    // → no recomposition, no relayout, just GPU-side transform per frame
}
```

7. Index as list key (or no key)

Bad:

```
LazyColumn {
    itemsIndexed(users) { index, user ->
        UserRow(user)                       // no key
    }
}
```

Why: Removing user #0 makes Compose think every other user changed (because each one shifted positions). Massive recomposition cascade.

Good:

```
LazyColumn {
    items(users, key = { it.id }) { user ->
        UserRow(user)                        // stable identity per row
    }
}
// Removing user #0 only animates that one row out; others are untouched.
```

8. Big `data class` with one var field destroying stability

Bad:

```
data class Profile(
    val id: String,
    val name: String,
    var lastSeen: Long                    // var → unstable
)
// Compose marks the whole class unstable; every Composable taking Profile loses skipping.
```

Good:

```
@Immutable
data class Profile(
    val id: String,
    val name: String,
    val lastSeen: Long                    // all val
)
// Or, if the property MUST be mutable, model it as State:
@Stable
class ProfileVm {
    var lastSeen by mutableStateOf(0L)
}
```

How to verify your fixes worked

1. **Layout Inspector → "Show recomposition counts"** — interact with the screen; counts should not climb absurdly. Anything > 10× higher than items appearing on screen is a smell.
2. **Compose Compiler metrics** (Gradle `composeCompiler.metricsDestination`) — generated `*-composables.txt` shows which Composables are `skippable` and `restartable`. Goal: every Composable that takes parameters should be both.
3. **Macrobenchmark** `FrameTimingMetric` — measure end-to-end frame time on a release build. Should be under 16.6 ms (60 Hz) or 8.3 ms (120 Hz).

The interview-cheat answer to "How do I avoid unnecessary recompositions?"

> Memorize this 5-step answer. Delivers in 30 seconds, demonstrates depth:
>
> 1. "I generate the Compose Compiler stability report — it tells me which Composables are skippable and which aren't, with reasons."
> 2. "For unstable parameters (List, Map, classes from other modules), I switch to `ImmutableList` from `kotlinx.collections.immutable` or annotate the type `@Immutable`."
> 3. "I defer State reads to the lowest level — read `name` in a leaf Text, not the whole UiState in a parent Container."
> 4. "For animated values, I move reads to the drawing phase via `Modifier.graphicsLayer { }` instead of value-form modifiers like `Modifier.rotate`."
> 5. "I always provide stable keys in lazy lists — entity ID, never index — and avoid passing inline lambdas that capture different values per row."

★ How the Compose Compiler transforms your code (the bytecode walkthrough)

The Compose Compiler is a Kotlin compiler plugin. Every `@Composable` function gets rewritten at compile time. This section walks through exactly what changes and why — the level of detail interviewers love.

Why a compiler plugin (and not a runtime library)?

Compose's runtime needs to know things about your code that ordinary Kotlin can't express:

- Which functions are restartable groups (so the runtime can re-execute them in isolation)
- Which parameters changed since last call (to decide skipping)
- Where each call is positioned in the source (for positional memoization)
- Which lambdas are stable (for memoization)

None of this can be expressed cleanly in plain Kotlin. The Compose Compiler runs as part of the Kotlin compilation pipeline (using Kotlin IR, the same intermediate representation that `suspend` uses). Same Kotlin compiler team that built suspend built Compose Compiler — and the transformations look very similar.

The four transformations applied to every @Composable

#	TRANSFORMATION	WHY
1	Inject hidden parameters: `$composer: Composer`, `$changed: Int`, `$default: Int`	The Composer threads runtime context; bitmasks track which params changed
2	Wrap body in `startRestartGroup(key)` / `endRestartGroup()`	Creates a RecomposeScope — a unit Compose can re-execute on its own
3	Add skip logic at function entry — early return if all params unchanged AND stable	This is "smart recomposition" in action; saves the body from running
4	Add `updateScope { c, _ -> ... }` at function exit	Tells the runtime how to re-invoke this function later (for invalidation)

Side-by-side: source → what the compiler emits

You write:

```
@Composable
fun Greeting(name: String) {
    Text("Hello $name")
}
```

Compiler emits (simplified, but conceptually accurate):

```
fun Greeting(
    name: String,
    $composer: Composer,
    $changed: Int
) {
    $composer.startRestartGroup(0xABCD)                  // (2) restart group with stable key

    // (3) skip-logic block — populate `dirty` bits and decide
    var dirty = $changed
    if ($changed and 0b1 == 0)
        dirty = dirty or if ($composer.changed(name)) 0b100 else 0b10

    if (dirty and 0b11 != 0b10 || !$composer.skipping) {
        // not skipping — actually run the body
        Text("Hello $name", $composer, 0)
    } else {
        // skipping — advance the slot table without running body
        $composer.skipToGroupEnd()
    }

    // (4) updateScope — captures how to re-invoke this function on invalidation
    $composer.endRestartGroup()?.updateScope { c, _ ->
        Greeting(name, c, $changed or 1)
    }
}
```

Three things to notice:

- The original 1-line function became ~15 lines of generated code. Compose runtime overhead is *per-Composable*, paid in compiled bytecode size, not in runtime allocation.
- Calling `Greeting("Sai")` in user code becomes `Greeting("Sai", $composer, $changed)` — the Composer is threaded through implicitly. You never see it.
- The early skip (`$composer.skipToGroupEnd()`) is the entire performance story. When skip happens, the body of the function literally doesn't run.

The `$changed` bitmask explained bit-by-bit

Each parameter gets **3 bits** in the `$changed` Int. So a Composable with up to 10 params packs everything into one Int (32 bits / 3 bits = 10 + spare bits).

BIT PATTERN (PER PARAM)	MEANING
`000` (0)	Unknown — caller didn't tell us, must check via `$composer.changed(value)`
`001` (1)	Same — value identical to last time, skip body
`010` (2)	Different — value changed since last time
`100` (4)	Static — value is a compile-time constant; trivially equal

The skip check is essentially: "for every param, are all bits saying it's unchanged AND the type is stable? If yes, skip."

How `updateScope` enables invalidation

The `updateScope` lambda captures the function reference + current arguments. When the snapshot system invalidates a State that this scope read, the Recomposer pulls this lambda out and invokes it — re-running the function with the same args, but the runtime now knows to re-fetch State values.

```
// What updateScope does conceptually:
$composer.endRestartGroup()?.updateScope { c, $force ->
    Greeting(name, c, $changed or 1)    // re-invoke with `or 1` = "force check params"
}
```

Without `updateScope`, Compose would have no way to re-run just this function — it would need to re-run the entire root. *This single piece of generated code is what makes targeted invalidation possible.*

Lambda memoization — why `onClick = { ... }` doesn't break skipping

Naive thinking: "every recomposition creates a new lambda; child Composable sees a different lambda each time and can't skip." That would be correct if the compiler didn't help.

The Compose Compiler runs an additional pass: **lambda memoization**. For lambdas with no captures or only stable captures, it wraps them in a `remember`-equivalent slot.

```
// You write:
@Composable fun Parent(vm: MyViewModel) {
    Button(onClick = { vm.click() }) { Text("Tap") }
}

// Compiler emits roughly:
@Composable fun Parent(vm: MyViewModel, $composer: Composer, $changed: Int) {
    $composer.startRestartGroup(...)
    val onClick = remember(vm) { { vm.click() } }   // memoized!
    Button(onClick, ...)
    $composer.endRestartGroup()?.updateScope { ... }
}
```

Now the same lambda instance is reused across recompositions — Button sees a stable parameter and stays skippable.

Caveat: this only works for lambdas with stable captures. If your lambda captures an unstable type, the compiler can't memoize it.

Default parameter handling — the `$default` bitmask

For `@Composable fun X(a: String = "x", b: Int = 0)`, the compiler emits an extra `$default: Int` parameter. Each bit indicates "this argument was supplied by caller (0)" or "use the default value (1)." This avoids the JVM's standard default-argument mechanism (which generates an extra dispatch method per default permutation) — keeps Compose call sites cheap.

Stability inference — the compiler's algorithm

For each class your code uses, the Compose Compiler decides one of three things:

- **Stable** — Compose can skip Composables taking this type when value is equal
- **Unstable** — must always recompose; never skip
- **Unknown** (rare) — falls back to runtime `$composer.changed(...)` check

The algorithm in plain English:

1. If the type is annotated `@Stable` or `@Immutable`, it's stable.
2. If the type is a primitive, String, or function type, it's stable.
3. If the type is a data class with all-stable properties AND all properties are `val`, it's stable.
4. If the type is from another module the compiler can't see fully, it's unstable.
5. If the type contains a `var`, an interface, or an unstable property, it's unstable.

Strong skipping mode (Compose Compiler 1.5.4+)

In strong skipping mode, even unstable parameters can be treated as skippable if their `equals()` returns true. This dramatically reduces stability bugs in practice. Enabled by default in Compose 1.6+.

Trade-off: `equals()` on a deep object graph is O(N) — for huge data classes the equality check might cost more than just recomposing.

How to read the compiler reports

Add to your Gradle file:

```
composeCompiler {
    metricsDestination = layout.buildDirectory.dir("compose-metrics")
    reportsDestination = layout.buildDirectory.dir("compose-reports")
}
```

After a release build, you get per-module files:

FILE	WHAT IT TELLS YOU
`*-classes.txt`	Stability of every class your module touches: `stable` / `unstable` / `runtime` + reasons
`*-composables.txt`	Per-Composable: `skippable`, `restartable`, `readonly`, plus stability of each parameter
`*-composables.csv`	Same data in CSV — easy to grep for "not skippable"
`*-module.json`	Module-level summary statistics

Goal: **every Composable that takes parameters should be both `skippable` AND `restartable`**. If a Composable is `restartable` but NOT `skippable`, find which parameter is unstable and fix it (per §9 of this guide).

> **The takeaway:** the Compose Compiler does ~5-10 transformations per `@Composable`, all aimed at one goal — making the runtime able to skip work. Your job as a developer is to hand the compiler the information it needs (stable types, immutable parameters, hoisted state) so its skip decisions can fire. Hand it bad inputs and skipping silently fails; hand it good inputs and your UI flies.

7. Side effects — picking the right effect API

Composables must be pure. To do anything that affects the outside world (network call, register a listener, start a timer), use a side-effect API. Compose ties these to the Composable's lifecycle so cleanup is automatic.

The decision tree

API	WHEN TO USE	CLEANS UP VIA
`LaunchedEffect(key1, key2, ...) { }`	Run a coroutine; cancel + restart when keys change	Coroutine cancellation when scope leaves composition or keys change
`DisposableEffect(key) { ... onDispose { } }`	Register/unregister listeners, observers, callbacks	`onDispose { }` block
`SideEffect { }`	Publish a Compose state value to non-Compose code (analytics, mutable singletons)	Runs after every successful recomposition
`rememberCoroutineScope()`	Launch coroutines from event handlers (button clicks, drag end)	Scope cancelled when Composable leaves composition
`produceState(initial, key) { value = ... }`	Convert a non-Compose async source to `State<T>`	Coroutine cancelled on scope leave / key change
`derivedStateOf { computed }`	Compute a value derived from other State; only recomposes readers when result changes	Tied to its enclosing `remember`
`rememberUpdatedState(value)`	Capture latest value in a long-running effect without restarting it	Reference; no cleanup

```
@Composable fun UserScreen(userId: String) {
    // LaunchedEffect — restart on userId change
    LaunchedEffect(userId) { vm.load(userId) }

    // DisposableEffect — register/unregister
    DisposableEffect(Unit) {
        analytics.startTracking()
        onDispose { analytics.stopTracking() }
    }

    // rememberCoroutineScope — launch from event handlers
    val scope = rememberCoroutineScope()
    Button(onClick = { scope.launch { vm.refresh() } }) { Text("Refresh") }
}
```

> **Don't:** call `LaunchedEffect` from inside an `onClick`. `LaunchedEffect` is a Composable, not a regular function — it must be in the Composable body. For event-driven launches use `rememberCoroutineScope`.

The classic `rememberUpdatedState` use case

```
@Composable fun Timer(onTimeout: () -> Unit) {
    // Capture latest onTimeout, but don't restart the timer when it changes
    val currentOnTimeout by rememberUpdatedState(onTimeout)
    LaunchedEffect(Unit) {
        delay(5_000)
        currentOnTimeout()           // always uses the latest version
    }
}
```

`8. derivedStateOf` — when to use it (and when not)

`derivedStateOf` wraps a computed value and only emits a new value when the *result* changes — even if its inputs change frequently.

```
@Composable fun ScrollAwareFab(listState: LazyListState) {
    // Bad: recomposes every scroll pixel — even when the boolean doesn't flip
    val showButton = listState.firstVisibleItemIndex > 10

    // Good: only recomposes when the boolean actually flips
    val showButton by remember {
        derivedStateOf { listState.firstVisibleItemIndex > 10 }
    }

    if (showButton) FloatingActionButton(...) { Icon(...) }
}
```

USE DERIVEDSTATEOF WHEN	DON'T USE IT WHEN
Inputs change far more often than the derived result (scroll position → "show button?")	The input changes infrequently — adds allocation + indirection for no gain
The derivation is expensive AND inputs are noisy	You're just transforming one State into another with the same change rate
You want a single flat boolean derived from multiple noisy sources	You're inside a non-Composable — `derivedStateOf` is a runtime construct, can be used outside Composables but typically paired with `remember`

> **Common mistake:** `derivedStateOf` without `remember` is rebuilt on every recomposition — same observation cost as plain reads. Always pair with `remember`.

9. Stability — why it matters and how to fix unstable types

"Stable" types are types Compose can prove won't change in surprising ways. Stable + unchanged parameters = the Composable can be **skipped** on recomposition. Unstable parameters = Compose conservatively re-runs the Composable.

What's stable by default

- Primitives (Int, Long, Boolean, Float, Double, Char) and Strings
- Function types (lambdas) — but only if their captures are stable
- `State<T>` and `MutableState<T>`
- Types annotated `@Stable` or `@Immutable`
- data classes whose properties are all stable types

What's *not* stable by default

- `List<T>`, `Set<T>`, `Map<K, V>` from kotlin.collections — they could be MutableList
- Interfaces (Compose can't see the implementation; assumes worst case)
- Classes with `var` properties
- Classes from other modules (compiler can't infer stability without seeing the source)

The two annotations

```
@Immutable
data class User(val id: String, val name: String)
// Promise: properties never change after construction. Compose treats as fully stable.

@Stable
class CounterStore {
    var count by mutableStateOf(0)
}
// Promise: properties may change, but I'll notify Compose via State. Compose treats as stable.
```

Strong skipping mode (Compose Compiler 1.5.4+)

Even unstable parameters can now be **skippable** if they pass an `equals()` check. Practical effect: most stability bugs disappear in Compose 1.6+. Still, mark types `@Immutable` when you can — it's clearer and handles cases where equals is O(N).

How to inspect stability

Add the Compose Compiler metrics flag to your Gradle config:

```
composeCompiler {
    metricsDestination = layout.buildDirectory.dir("compose-metrics")
    reportsDestination = layout.buildDirectory.dir("compose-reports")
}
```

After a release build, the reports directory contains per-module `*-classes.txt` (stability of every class) and `*-composables.txt` (skippable / restartable status of every Composable). Best perf-debugging tool you can ship to your team.

Common stability fixes

- Use `kotlinx.collections.immutable` — `ImmutableList<T>`, `PersistentList<T>` are `@Immutable`.
- Make data class properties `val`, not `var`.
- Annotate domain models `@Immutable`.
- For interface parameters, mark them `@Stable` on the interface itself or replace with sealed classes.

10. `key()` and list `itemKey` — identity in lists

By default, Compose identifies items by **position** in the call tree. For lists where items can be added, removed, or reordered, you must give Compose a stable identity, or it'll associate state with the wrong item.

```
LazyColumn {
    items(
        items = users,
        key = { user -> user.id }              // stable identity
    ) { user ->
        UserRow(user)
    }
}

// Inside a non-list Composable: key() block
users.forEach { user ->
    key(user.id) {                             // scoped identity
        UserRow(user)
    }
}
```

Symptoms of missing keys:

- State stuck on the wrong item after add/remove (e.g., a checkbox state moves)
- Janky animations that look like rebuilds rather than moves
- `rememberSaveable` values appearing on the wrong rows

> **Don't:** use the index as the key. That's the same as no key — defeats the whole purpose. Don't use the entire object either; equals must be cheap and identity-meaningful.

11. Modifiers — order matters; `Modifier.Node` for performance

A `Modifier` is an immutable, ordered chain of behaviors applied to a Composable: padding, click, background, etc. **Order is meaningful** — each modifier wraps the next.

```
// 16 dp padding, then background → padding outside the color
Box(Modifier.padding(16.dp).background(Color.Red))

// background, then padding → padding inside the color
Box(Modifier.background(Color.Red).padding(16.dp))
```

Common modifier reading order

1. Size (`fillMaxWidth`, `size`, `height`) defines the constraints
2. Padding adds inset
3. Background paints behind contents
4. Border draws on top of background
5. Click areas use the modified bounds

Custom modifiers — use `Modifier.Node`

```
// Modern (recommended) — Modifier.Node + ModifierNodeElement
private class MyHighlightNode : Modifier.Node(), DrawModifierNode {
    override fun ContentDrawScope.draw() {
        drawRect(Color.Yellow.copy(alpha = 0.3f))
        drawContent()
    }
}

private data object MyHighlightElement : ModifierNodeElement<MyHighlightNode>() {
    override fun create() = MyHighlightNode()
    override fun update(node: MyHighlightNode) {}
}

fun Modifier.highlight() = this then MyHighlightElement
```

Why `Modifier.Node` over the older `composed { }`:

- Reusable across recompositions — fewer allocations
- Doesn't trigger recomposition just because the modifier was rebuilt
- Can hook into the layout, draw, focus, pointer-input phases via specialized interfaces (DrawModifierNode, LayoutModifierNode, etc.)

> **Don't:** use `composed { }` for new code. It's flagged as deprecated in newer Compose versions and creates an extra Composable per call site.

12. The three phases — Composition, Layout, Drawing

Compose runs three phases per frame, top-down. Understanding which phase a State read happens in determines what gets invalidated when the State changes.

PHASE	WHAT HAPPENS	STATE READ HERE INVALIDATES
Composition	Run @Composable functions, build the tree (slot table + LayoutNodes)	Composition + Layout + Drawing
Layout	Measure each node (with constraints from parent), then place children	Layout + Drawing
Drawing	Paint pixels into the buffer	Drawing only

Defer state reads to the latest possible phase

```
// Bad — reads scroll value in Composition phase → recomposes on every scroll pixel
Text("Offset: ${scrollState.value}")

// Good — reads in Layout phase via lambda → only relayouts, no recomposition
Box(Modifier.offset { IntOffset(0, scrollState.value) })

// Even better for transforms — reads in Drawing phase → only redraws
Box(Modifier.graphicsLayer { translationY = scrollState.value.toFloat() })
```

Modifiers that take a lambda (`Modifier.offset { }`, `Modifier.graphicsLayer { }`, `Modifier.drawBehind { }`) defer execution to a later phase. Reading State inside the lambda doesn't trigger recomposition — only the relevant later-phase invalidation.

13. Custom `Layout` and `SubcomposeLayout`

For most cases, built-in containers (`Box`, `Row`, `Column`, `LazyColumn`) are enough. When you need custom measurement/placement, drop to `Layout`.

```
@Composable
fun SpacedColumn(spacing: Dp, content: @Composable () -> Unit) {
    Layout(content = content) { measurables, constraints ->
        val placeables = measurables.map { it.measure(constraints) }
        val spacingPx = spacing.roundToPx()
        val totalH = placeables.sumOf { it.height } + spacingPx * (placeables.size - 1)
        layout(constraints.maxWidth, totalH) {
            var y = 0
            placeables.forEach { it.placeRelative(0, y); y += it.height + spacingPx }
        }
    }
}
```

`SubcomposeLayout` — when you need measurement to influence composition

Used internally by `Scaffold`, `BoxWithConstraints`, `LazyColumn`. Lets you compose children based on measurement results from *other* children.

```
SubcomposeLayout { constraints ->
    val headerPlaceable = subcompose("header") { Header() }
        .first().measure(constraints)
    val remaining = constraints.copy(maxHeight = constraints.maxHeight - headerPlaceable.height)
    val bodyPlaceable = subcompose("body") { Body() }
        .first().measure(remaining)
    layout(constraints.maxWidth, constraints.maxHeight) {
        headerPlaceable.placeRelative(0, 0)
        bodyPlaceable.placeRelative(0, headerPlaceable.height)
    }
}
```

> **Don't:** reach for `SubcomposeLayout` if a regular `Layout` works. SubcomposeLayout composes during measure, which disables intrinsics and adds significant cost.

14. `CompositionLocal` — implicit values down the tree

A way to pass values implicitly through the Composable tree without threading them through every parameter. Built-in examples: `LocalContext`, `LocalDensity`, `LocalConfiguration`, `MaterialTheme.colorScheme`.

```
val LocalAppUser = compositionLocalOf<User> { error("No User provided") }

@Composable fun App() {
    val currentUser = vm.user.collectAsStateWithLifecycle().value
    CompositionLocalProvider(LocalAppUser provides currentUser) {
        Screen()
    }
}

@Composable fun Avatar() {
    val user = LocalAppUser.current      // implicit lookup
    Image(user.avatar, contentDescription = user.name)
}
```

Two flavors

- `compositionLocalOf` — only direct readers recompose when the value changes (more granular)
- `staticCompositionLocalOf` — entire subtree recomposes on change. Cheaper at runtime if the value rarely changes (Theme, Density)

USE COMPOSITIONLOCAL FOR	DON'T USE FOR
Cross-cutting plumbing: theme, density, navigation controller, image loader	App-level domain state — use a ViewModel
Values needed deep but rarely	Values used by 90% of Composables — just pass as parameter
Hosting Java/legacy values that don't fit Compose state	"Avoiding boilerplate" — too much CompositionLocal makes data flow opaque

15. Lists — `LazyColumn` and friends

Lazy components only compose visible items + a small pre-fetch window. They handle thousands of items efficiently.

```
LazyColumn(
    state = rememberLazyListState(),
    contentPadding = PaddingValues(16.dp),
    verticalArrangement = Arrangement.spacedBy(8.dp)
) {
    item { Header() }
    items(
        items = users,
        key = { it.id },                        // stable identity (§10)
        contentType = { it.javaClass }           // improves view-recycling
    ) { user ->
        UserRow(user)
    }
    item { Footer() }
}
```

Performance rules for lists

- Always provide `key`. Always.
- Provide `contentType` when items can have different shapes — Compose recycles by type.
- Don't read `listState.firstVisibleItemIndex` in Composition unless wrapped in `derivedStateOf` (§8).
- Use `animateItemPlacement()` on item Composables for free reorder animations.
- Hoist heavy state outside the item Composable; the item function should be lightweight.

Other lazy containers

CONTAINER	USE FOR
`LazyColumn` / `LazyRow`	1D linear lists
`LazyVerticalGrid` / `LazyHorizontalGrid`	2D grids with rectangular cells
`LazyVerticalStaggeredGrid`	Pinterest-style irregular grids
`HorizontalPager` / `VerticalPager`	Page-snapping (Stories, onboarding)

16. Animations — high-level → low-level

API	USE FOR
`animate*AsState` (`animateColorAsState`, `animateDpAsState`...)	Single-value animation; smooth interpolation when target changes
`updateTransition`	Coordinate multiple animated values driven by a single state
`AnimatedVisibility`	Enter/exit a Composable with built-in transition specs
`AnimatedContent`	Swap between two Composables with cross-fade-style transition
`animateContentSize()`	Animate size when content reflows
`rememberInfiniteTransition`	Looping animations (loading spinners, pulses)
`Animatable`	Imperative animation control (start/stop/snap from event handlers)
`Modifier.graphicsLayer { }`	GPU-accelerated transforms (rotation, scale, alpha) without recomposition

```
// Single value — most common
val alpha by animateFloatAsState(if (visible) 1f else 0f)

// Coordinated multi-value
val transition = updateTransition(state, label = "main")
val color    by transition.animateColor { if (it == State.A) Red else Blue }
val radius   by transition.animateDp    { if (it == State.A) 12.dp else 36.dp }

// graphicsLayer — read State during DRAWING phase, no recomposition
Box(Modifier.graphicsLayer {
    rotationZ = animatedAngle
    alpha = animatedAlpha
})
```

> **Performance trick:** for any property that animates frequently (rotation, alpha, scale), prefer `graphicsLayer { }` over the equivalent modifier (`Modifier.rotate`, `Modifier.alpha`). The lambda runs in the drawing phase, so the animation skips composition AND layout.

17. Theming — Material 3, dynamic color, custom design systems

```
@Composable
fun AppTheme(content: @Composable () -> Unit) {
    val dark = isSystemInDarkTheme()
    val colors = when {
        Build.VERSION.SDK_INT >= 31 ->             // Android 12+ dynamic color
            if (dark) dynamicDarkColorScheme(LocalContext.current)
            else dynamicLightColorScheme(LocalContext.current)
        dark -> darkColorScheme()
        else -> lightColorScheme()
    }
    MaterialTheme(
        colorScheme = colors,
        typography = AppTypography,
        shapes = AppShapes,
        content = content
    )
}
```

Custom design system on top of MaterialTheme

If you have brand-specific tokens beyond Material's color/typography/shape:

```
data class AppColors(val brandRed: Color, val warningYellow: Color)
val LocalAppColors = staticCompositionLocalOf<AppColors> { error("No AppColors") }

object AppTheme {
    val colors: AppColors
        @Composable
        get() = LocalAppColors.current
}

@Composable fun Header() {
    Text("Hello", color = AppTheme.colors.brandRed)
}
```

18. Navigation — type-safe routes (Nav 2.8+)

Compose Navigation 2.8+ uses Kotlin Serialization for type-safe destinations — pass data class objects, not strings.

```
@Serializable data object Home
@Serializable data class Profile(val userId: String, val tab: Int = 0)

NavHost(navController, startDestination = Home) {
    composable<Home> { HomeScreen(onProfileClick = { navController.navigate(Profile("42")) }) }
    composable<Profile> { entry ->
        val args: Profile = entry.toRoute()
        ProfileScreen(args.userId, args.tab)
    }
}

navController.navigate(Profile(userId = "42", tab = 1))
```

Sharing a ViewModel between screens (nested graph)

```
navigation<CheckoutGraph>(startDestination = CartScreen) {
    composable<CartScreen> { entry ->
        val parent = remember(entry) {
            navController.getBackStackEntry<CheckoutGraph>()
        }
        val sharedVm: CheckoutViewModel = hiltViewModel(parent)
        ...
    }
    composable<PaymentScreen> { entry ->
        val parent = remember(entry) {
            navController.getBackStackEntry<CheckoutGraph>()
        }
        val sharedVm: CheckoutViewModel = hiltViewModel(parent)
        ...
    }
}
```

Deep links

```
composable<Profile>(
    deepLinks = listOf(navDeepLink<Profile>(basePath = "https://app.com/u"))
) { ... }
```

19. Edge-to-edge & `WindowInsets`

Edge-to-edge = your app draws under the system bars (status, navigation, IME, cutout). Mandatory in Android 15+ for apps targeting SDK 35.

```
// In Activity
override fun onCreate(b: Bundle?) {
    super.onCreate(b)
    enableEdgeToEdge()                    // from androidx.activity:activity
    setContent { App() }
}

// In Compose — apply only the insets you need
Scaffold(
    contentWindowInsets = WindowInsets.safeDrawing
) { padding ->
    Column(Modifier.padding(padding)) { ... }
}

// Or per-surface
Surface(Modifier.statusBarsPadding()) { TopBar() }
Surface(Modifier.navigationBarsPadding()) { BottomBar() }
TextField(Modifier.imePadding(), ...)
```

INSET	COVERS
`statusBars`	Top status bar (clock, battery)
`navigationBars`	Bottom 3-button or gesture bar
`ime`	Keyboard area when shown
`displayCutout`	Notch / camera cutout
`safeDrawing`	Status + nav + IME + cutout (composite)
`safeContent`	safeDrawing minus IME
`safeGestures`	Avoid system gesture-only zones (back, home swipes)

Common pitfall: applying `safeDrawing` at the very root when you have a colored TopAppBar leaves an unwanted gap above the bar. Apply insets per surface instead — let the TopAppBar use `statusBarsPadding` internally; let the body use what it needs.

20. Interop — `AndroidView` and `ComposeView`

Hosting a View inside Compose

```
AndroidView(
    factory = { ctx ->
        MapView(ctx).apply {
            onCreate(null)
            getMapAsync { map -> ... }
        }
    },
    update = { mapView ->
        // Re-runs on recomposition; receive new state
        mapView.invalidate()
    }
)
```

Hosting Compose inside XML

```
<androidx.compose.ui.platform.ComposeView
    android:id="@+id/compose_root"
    android:layout_width="match_parent"
    android:layout_height="wrap_content" />
```

```
binding.composeRoot.setContent { MyComposable() }
```

	USE WHEN
`AndroidView`	Wrapping legacy custom Views, Maps SDK, AdView, third-party Views without Compose support
`ComposeView`	Migrating an XML-based screen incrementally — drop a Composable into a corner
Pure Compose	Default for new screens. Reach for interop only when needed.

> **Tip:** `AndroidView` is heavy compared to native Composables. The View lifecycle is tied to composition lifecycle; mount/unmount can churn. Pass a `key` argument that's stable across recompositions.

21. Coroutines & Flow integration

Collect Flow safely (lifecycle-aware)

```
@Composable fun Screen(vm: MyViewModel = hiltViewModel()) {
    val state by vm.uiState.collectAsStateWithLifecycle()    // pauses in background
    Render(state)
}
```

API	BEHAVIOR
`collectAsState()`	Collects regardless of lifecycle — drains battery in background. Avoid in 2026.
`collectAsStateWithLifecycle()`	Pauses collection when lifecycle is below STARTED. **Default in 2026.**
`livedata.observeAsState()`	For LiveData — already lifecycle-aware via Compose's lifecycle integration.

Convert async source to State — `produceState`

```
val user: State<User?> = produceState<User?>(initialValue = null, key1 = userId) {
    value = api.getUser(userId)
    // Coroutine cancelled on Composable leaves composition or userId change
}
```

Launch from event handlers — `rememberCoroutineScope`

```
val scope = rememberCoroutineScope()
Button(onClick = { scope.launch { vm.refresh() } }) { Text("Refresh") }
```

22. Testing — fast, deterministic Compose tests

```
@get:Rule
val compose = createComposeRule()                // for non-Hilt
// or createAndroidComposeRule<ActivityT>()        for AndroidTest with Activity

@Test fun counter_incrementsOnClick() {
    compose.setContent { CounterScreen() }
    compose.onNodeWithText("Clicks: 0").assertIsDisplayed()
    compose.onNodeWithText("Click me").performClick()
    compose.onNodeWithText("Clicks: 1").assertIsDisplayed()
}
```

HELPER	USE FOR
`onNodeWithText("X")`	Find by visible text
`onNodeWithTag("submit")`	Find by `Modifier.testTag(...)` — robust across i18n
`onNodeWithContentDescription(...)`	Find by accessibility description
`performClick()` / `performTextInput(...)` / `performScrollTo(...)`	Drive interactions
`assertIsDisplayed()` / `assertIsEnabled()` / `assertTextEquals(...)`	Assertions
`compose.waitForIdle()`	Wait for async work to settle
`compose.mainClock.autoAdvance = false; mainClock.advanceTimeBy(ms)`	Control time for animation tests

Screenshot testing — Paparazzi / Roborazzi

Render Composables to PNG without an emulator; diff against golden images. Catches visual regressions instantly. Both libraries integrate cleanly with the design-system testing flow.

23. Accessibility — TalkBack, semantics, touch targets

Compose builds a semantics tree separate from the visual tree. TalkBack reads it. Add semantic info via `contentDescription`, click labels, and the `semantics { }` modifier.

```
// Decorative image — explicitly null content description
Image(painterResource(R.drawable.bg), contentDescription = null)

// Meaningful image
Image(painterResource(R.drawable.profile), contentDescription = "Profile picture")

// Custom click — TalkBack announces "Double-tap to like"
Box(Modifier.clickable(onClickLabel = "Like this post") { vm.like() })

// Group descendants into one semantic node
Row(Modifier.semantics(mergeDescendants = true) {}) {
    Icon(...)
    Text("Settings")
}

// Custom semantic state for a custom slider
Modifier.semantics {
    contentDescription = "Volume"
    progressBarRangeInfo = ProgressBarRangeInfo(0.7f, 0f..1f)
    role = Role.Slider
}
```

RULE	THRESHOLD
Touch target size	≥ 48 dp (Material guideline; Accessibility Scanner flags smaller)
Color contrast (normal text)	4.5 : 1
Color contrast (large text)	3 : 1
Don't rely on color alone	Always pair color cues with icons or text

24. Performance — finding and fixing recompositions

How to find offending recompositions

1. **Layout Inspector → "Show recomposition counts"** — see counts per Composable as you interact.
2. **Compose Compiler metrics report** (§9) — per-Composable skippable / restartable info; per-class stability.
3. **Macrobenchmark + FrameTimingMetric** — measure end-to-end frame timing on a release build.
4. **Perfetto trace** — low-level system trace; see scheduling and main-thread work.

Common fixes

- **Stable parameters**: replace `List<T>` with `ImmutableList<T>` from kotlinx.collections.immutable.
- **Defer State reads to later phases**: use lambda-modifiers (`Modifier.offset { }`, `graphicsLayer { }`) instead of value-modifiers.
- **Wrap derived values**: `derivedStateOf` for computed booleans on noisy inputs.

- **Cache lambdas**: in scroll callbacks, `val onClick = remember { { vm.click() } }` avoids new Function instances per frame.
- **Stable keys for lists**: never index, always entity ID.
- **Ship a Baseline Profile**: pre-AOT-compile your hot paths. Compose-heavy apps see 20-40% cold-start improvement.
- **Use primitive State**: `mutableIntStateOf` over `mutableStateOf(0)` — avoids boxing.

> **Don't profile debug builds.** Compose generates lots of extra debug-only allocations. Always profile a release build with R8 + Baseline Profile installed.

25. Do's and Don'ts — comprehensive checklist

DO

- Make UI a function of state. UI = f(state).
- Hoist state to the lowest common ancestor of Composables that need it.
- Use `collectAsStateWithLifecycle` for Flow in Compose.
- Provide stable keys for every list.
- Mark domain models `@Immutable`.
- Use `derivedStateOf` for noisy inputs producing rare boolean outputs.
- Use `graphicsLayer { }` lambda for animated transforms.
- Use `Modifier.testTag` for stable test selectors.
- Run release build + Baseline Profile when measuring perf.
- Provide `contentDescription` for meaningful images; `null` for decorative.
- Use `rememberSaveable` for UI state that must survive process death.
- Pair side effects with the right effect API (`LaunchedEffect`, `DisposableEffect`).

DON'T

- Mutate plain `var` from inside a Composable expecting recomposition.
- Run network calls or I/O directly in the Composable body.
- Use `collectAsState()` in production — wastes battery in background.
- Use the index as a list key.
- Call `LaunchedEffect` from an `onClick`.
- Forget `remember` around `derivedStateOf` / `mutableStateOf`.
- Pass mutable `List<T>` to Composables and expect skipping.
- Read `scrollState.value` in Composition without `derivedStateOf`.
- Use `SubcomposeLayout` when a regular `Layout` works.
- Mass-apply `safeDrawing` at the root over a colored TopAppBar.
- Profile debug builds for performance work.
- Ignore the Compose Compiler metrics report — it's the truth.

26. Compose vs XML/Views — when to pick which

	COMPOSE (2026)	VIEWS/XML
Code volume per typical screen	~50% less	baseline
State sync	Automatic (reactive)	Manual (mutate then notify)
Custom layout	~tens of lines (custom `Layout`)	Hundreds (custom `ViewGroup`)
Animations	First-class (`animate*AsState`, `graphicsLayer`)	Animator + ObjectAnimator
Live preview	`@Preview` in Studio	Layout Inspector after launch
Maturity	Solid in 2026; few niche gaps (advanced gestures, complex map embeds)	15+ years
Cold start cost	Slightly more without Baseline Profile; closes the gap with one	Lighter at startup
Interop	Bidirectional via `AndroidView` / `ComposeView`	n/a
Hiring market (2026)	Default for new Android positions	Legacy maintenance roles

When to still pick Views

- 100% Java codebase with no Kotlin migration plan
- Existing XML-based screens that work — don't refactor for refactor's sake
- Heavy dependence on third-party Views without Compose wrappers (some IAP SDKs, ads)
- Team has zero Compose experience and a launch in 4 weeks

When to definitely pick Compose

- Any new screen in any modern Android codebase
- Anything with complex dynamic content (chat, feed, animated UI)
- Custom UI that would need `ViewGroup` in the View world
- Apps targeting Compose Multiplatform (KMP-shared UI)

27. One-page recall card

The 30-second mental model

Compose = declarative UI. UI = f(state). Compose Compiler rewrites every `@Composable` to inject a Composer + change-tracking bitmask + restart group. Composer maintains the slot table. Snapshot system tracks State reads per RecomposeScope. State change → invalidates the smallest enclosing scope → next frame, that scope re-runs.

State

- `mutableStateOf`, `mutableIntStateOf` for primitives
- `remember { }` for in-composition; `rememberSaveable { }` for process death
- `derivedStateOf { }` + `remember` for noisy inputs → rare output
- State hoisting: pass `(value, onChange)`, hoist to lowest common ancestor

Side effects

- `LaunchedEffect(key)` — coroutine, restart on key change
- `DisposableEffect(key)` — register/unregister with `onDispose`
- `SideEffect` — publish to non-Compose code
- `rememberCoroutineScope()` — launch from event handlers
- `produceState` — async source → State
- `rememberUpdatedState` — capture latest in long-running effect without restart

Performance

- Stable parameters → skipping; mark with `@Immutable`
- `ImmutableList<T>` over `List<T>`
- Defer State reads to Layout/Drawing via lambda-modifiers
- `graphicsLayer { }` for animated transforms
- Stable keys in lists (entity ID, never index)
- Compose Compiler metrics report = ground truth
- Baseline Profile for cold start

Internals (the engine)

- Compiler injects `Composer` + `$changed` bitmask
- Body wrapped in `startRestartGroup` / `endRestartGroup` + `updateScope`
- Slot table = flat gap-buffer array of all groups, parameters, remembered values
- Positional memoization = `remember` uses call-site position as the key
- Snapshot system = MVCC; read tracking via thread-local SnapshotObserver
- RecomposeScope = smallest re-runnable unit; State change invalidates the scope that read it
- Recomposer = singleton coroutine on AndroidUiDispatcher; batches per frame

Avoid

- `collectAsState()` (use lifecycle variant)
- Index as list key
- Plain `var` for UI state
- Side effects in Composable body
- `LaunchedEffect` inside `onClick`
- Profiling debug builds
- SubcomposeLayout when Layout works
- Mutable `List<T>` as Composable parameter

Read end-to-end for understanding · skim §25 (Do's and Don'ts) and §27 (recall card) the night before any Android interview. Pair with the **Top 210 Android Interview Questions** guide and the **Mobile System Design Playbook** for full coverage.